

PS-021: Experimental Dialog Correlation and Recovery Across Agent Adapters

Two mock agent adapters that keep task identity and never repeat recorded work — across lost requests, lost replies and restarts.

Experimental prototype · not exactly-once · not MCP/A2A · not an IETF standard

The problem

- Agents hand tasks to each other across frameworks; each tracks "which conversation is this?" its own way.
- Mid-task, a **request** or a **reply** gets lost, or an agent **restarts**.
- The sender cannot tell whether the work was done — so it retries.
- **Lost reply + retry** → the work runs twice (two messages, two records, charged twice).
- **Restart** with state only in memory → the task is forgotten; a retry looks like new work.
- PS-021 asks for: dialog IDs, explicit lifecycle states, request deduplication, durable recovery — tested on five disconnect/retry scenarios.

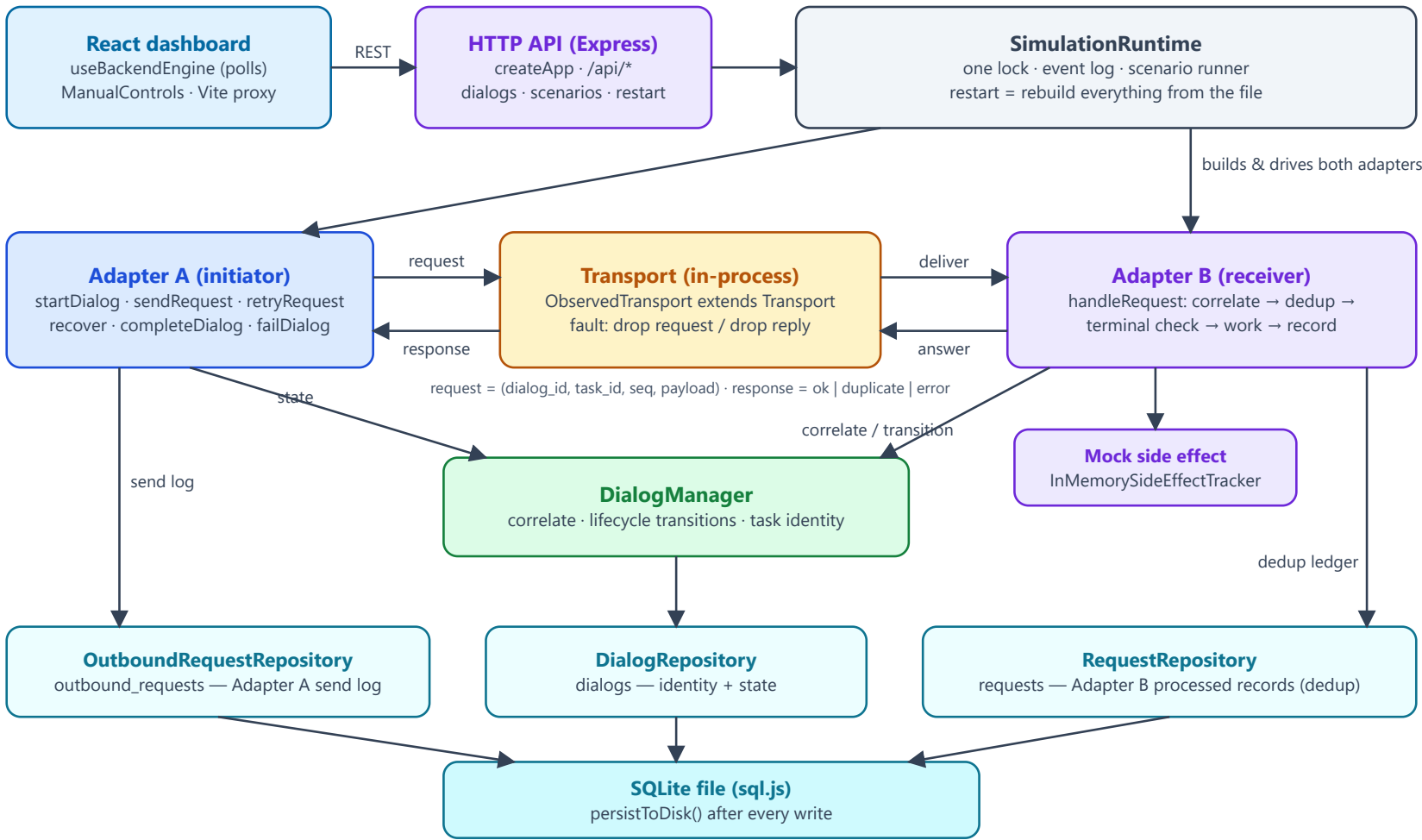
Approach: name it, write it down, check before acting

Identifier	Meaning	Where it lives
<code>dialog_id</code>	One A↔B exchange about one task — the correlation key	<code>dialogs</code> table (with <code>state</code> , <code>restored</code>)
<code>task_id</code>	The unit of work; B rejects a request whose <code>task_id</code> differs	<code>dialogs</code> table
<code>seq</code>	One logical request; <code>(dialog_id, seq)</code> = the dedup key	A's send log <code>outbound_requests</code> · B's records <code>requests</code>

- **Write it down first:** A logs each request `PENDING` before sending; every write goes to the SQLite file.
- **Check before acting:** B returns the *stored* result if `(dialog_id, seq)` was already processed.
- Schema: clearly labelled `v0.1-experimental` (no mentor draft pinned) — `docs/EXPERIMENTAL_SCHEMA.md`.

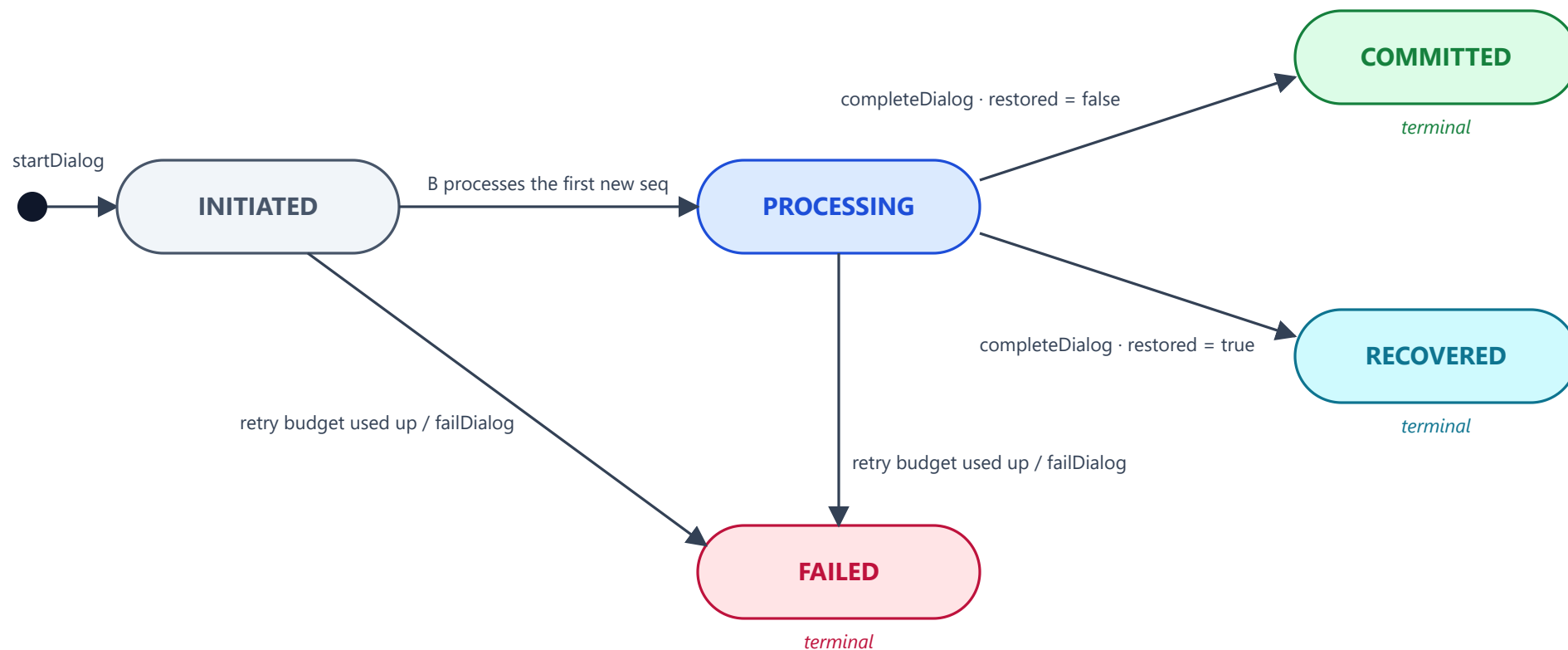
Architecture

Nighthawks PS-021 — architecture (one Node.js process, one SQLite file)



Lifecycle — exactly five states

Lifecycle — exactly the five backend states (backend/src/types/index.ts)



`completeDialog` needs every seq ACKED (otherwise 409 PENDING_REQUESTS); retry budget = `maxAttempts` (default 5).
Terminal states accept no new work; a retry of an already-processed seq still returns the stored answer.

The five disconnect / retry scenarios

#	Scenario	What fails	Outcome
1	Multiple Dialogs + Retry → Correct Correlation	D2's request lost (D1 also open)	Retry lands on D2 only; D1 untouched — D2 COMMITTED
2	Request Lost → Retry	Request never reaches B	Processed once on the retry — COMMITTED
3	Response Lost → Duplicate Request	B did the work, reply lost	Retry = duplicate, stored answer, work not repeated
4	Adapter B Restart → Durable State Recovery	Reply lost, then restart	Reloaded (next seq 2, pending [1]); retry = duplicate — RECOVERED
5	Mid-Task Disconnect + Adapter A Restart → Resume	seq 3 reply lost, then restart	next seq 4 from the file; same ids; continues — RECOVERED

Each runs as an adapter-level test, on the live runtime, as an HTTP call sequence, from the run endpoint, and from a dashboard card.

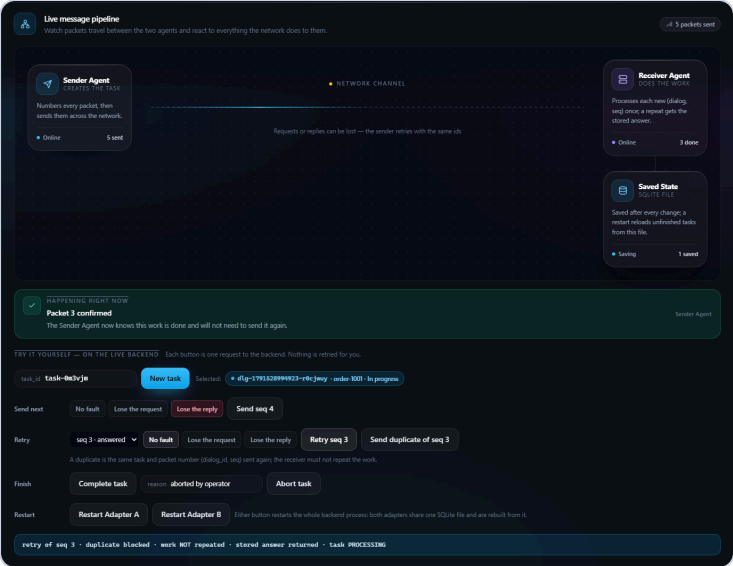
Live demo

```
node scripts/demo.mjs  
node scripts/trace.mjs
```

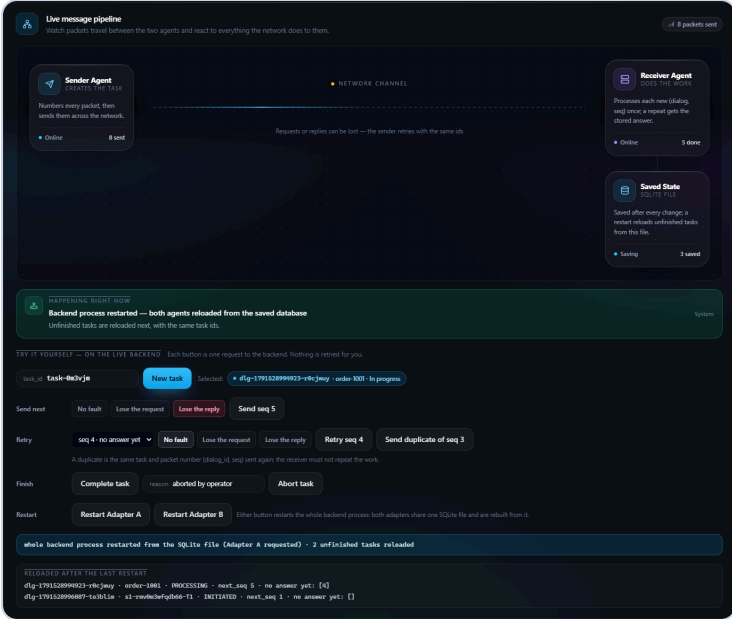
Dashboard on :5173, backend on :3001 with a fresh demo database;
the trace prints one line per backend event.

- Start a task → normal request
- Request lost → retry → processed once
- Reply lost → retry → **duplicate blocked, work NOT repeated**
- Scenario 1: correlation across two dialogs
- Restart Adapter A → same ids, next_seq, pending seq → **RECOVERED**
- Optional: kill the real backend process, restart on the same file
- Run all five → 5 of 5 passed

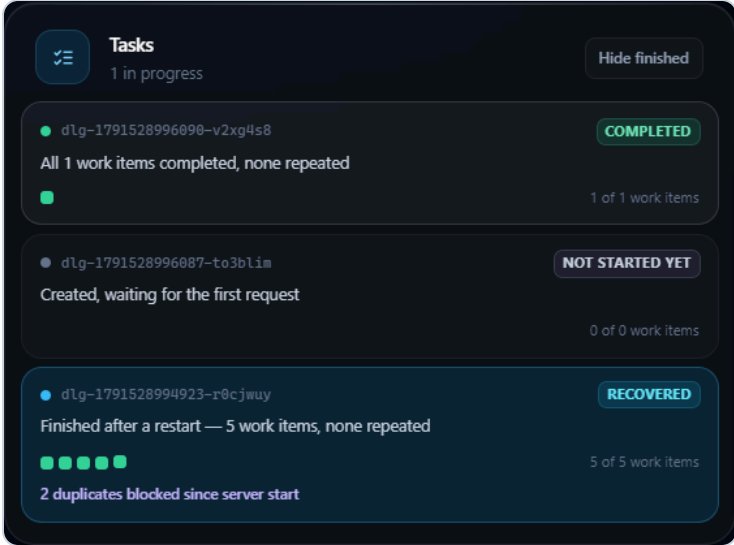
Recovery, as the dashboard shows it



Reply lost → retry: **duplicate blocked** · work **NOT repeated** · **stored answer returned**



After Restart Adapter A: same dialog_id / task_id, **next_seq 5**, seq 4 still unanswered



Task ends **RECOVERED** — "5 work items, none repeated"

Test evidence (2026-10-09)

Level	Count	Result
Backend unit + integration (Vitest)	265 tests · 14 files	all passed
— of which HTTP API (supertest)	62 tests	all passed
Frontend unit (Node + jsdom)	65 checks	all passed
Client smoke vs a real backend	6 steps	all passed
Browser E2E (Playwright + Edge, real backend + SQLite)	16 tests	all passed, repeated runs

Mutation check — breaking the dedup check in Adapter B fails 10 of 16 browser tests; breaking the duplicate result line fails 3; breaking reconnect fails 1. Details: `docs/TEST_RESULTS.md`.

Pseudocode — Adapter B, in this order

```
handleRequest(req):                                     # backend/src/adapters/AdapterB.ts
  dialog = correlate(req.dialog_id, req.task_id)        # DIALOG_NOT_FOUND / TASK_MISMATCH
  if processed.has(req.dialog_id, req.seq):
    return duplicate(stored result)                    # work NOT re-run
  if dialog is terminal: return error(DIALOG_TERMINAL)
  result = run_side_effect(req.payload)                # — crash window opens
  processed.insert(req.dialog_id, req.seq, result)      # persisted — crash window closes
  if dialog.state == INITIATED: transition(PROCESSING)
  return ok(result)

recover():                                              # AdapterA.ts — after every restart
  for each INITIATED / PROCESSING dialog: restored = true,
    next_seq = send_log.max_seq + 1, pending_seqs = send_log.pending
```

Limitations & non-claims

- **Not exactly-once** — a recorded request is never re-run; a crash between the work and its record can repeat it.
 - **Not an IETF standard** — experimental schema, inspired by agentproto.
 - **Not MCP / A2A**; no interoperability with them.
 - **No interoperability between independently built adapters** claimed — pending mentor clarification.
- One process, one shared SQLite file (restart A = restart B).
 - File rewritten in place — a crash during the write could corrupt it.
 - In-process transport: drop request / drop reply only.
 - Side-effect / duplicate counters and the event log are in memory.
 - Local, unauthenticated demo API.

Team Nighthawks · deliverables

Team members: <to be completed by the team>

Repository: github.com/anuragyy403/Nighthawks

- Report — docs/submission/Nighthawks-PS-021-Report.pdf
- Schema — docs/EXPERIMENTAL_SCHEMA.md
- Pseudocode — docs/PSEUDOCODE.md
- Test results — docs/TEST_RESULTS.md
- Demo script — docs/DEMO_SCRIPT.md
- Demo video — `cd e2e && npm run demo:record`